

RTOS-MovEdge

Clasificador de movimientos

Descripción de la Actividad.

Como hemos visto, **un clasificador de movimientos** es un sistema que identifica y categoriza distintos tipos de movimientos o gestos realizados por un objeto o persona basándose en datos de entrada, usualmente provenientes de sensores. En general, utilizan técnicas de procesamiento de señales y/o aprendizaje automático para analizar los datos sensoriales y asignar cada muestra de datos a una categoría de movimiento específica. Estos clasificadores son fundamentales en aplicaciones donde es crucial interpretar o monitorear la actividad física, como en el deporte, la rehabilitación médica, la robótica, entre otros.

Funcionamiento general:

- * **Recopilación de Datos:** El clasificador comienza con la recopilación de datos de movimiento, generalmente a través de sensores como acelerómetros, giroscopios, o cámaras. Estos dispositivos capturan la dinámica del movimiento en forma de datos crudos que pueden incluir aceleración, orientación, posición, entre otros.
- * **Preprocesamiento:** Los datos recogidos se procesan para eliminar el ruido y extraer características relevantes que describen los movimientos de manera efectiva. Esto puede incluir la normalización, el filtrado, o la transformación de los datos.
- * **Extracción de Características:** Se extraen características significativas de los datos preprocesados, como la amplitud máxima, la frecuencia dominante, o los valores medios. Estas características ayudan a distinguir entre diferentes tipos de movimientos.
- * **Entrenamiento del Modelo:** Se utiliza un conjunto de datos etiquetado para entrenar un modelo de aprendizaje automático. Este modelo aprende a asociar patrones de características con categorías específicas de movimiento.
- * **Clasificación:** Una vez entrenado, el modelo puede clasificar nuevos datos de movimiento en tiempo real o en lotes, asignando cada segmento de datos a una categoría de movimiento basada en sus características.

En esta actividad práctica realizaremos la implementación de un Clasificador de movimientos realizando la recopilación de datos de un acelerómetro y giroscopio. En este segundo caso de estudio realizaremos las etapas de recopilación de datos, preprocesamiento, extracción de características, y la implementación de un clasificador basado en funciones de probabilidad del tipo probit al igual que el uso del freeRTOS en

la implementación de una solución en la tarjeta de desarrollo ESP32.

Resultados de aprendizaje

Esta actividad ofrece una oportunidad para explorar el uso de tarjetas de desarrollo como la ESP32 y el uso del FreeRTOS. Los estudiantes aprenderán a dar solución a un sistema de clasificación usando las librerías del FreeRTOS. Al final de la actividad, los estudiantes deberían ser capaces de:

- * Definir diferentes tareas dentro del FreeRTOS y asignar prioridades que permitan ejecutar diferentes actividades dentro de la solución de un clasificador de movimientos.
- * Definir formas de comunicar las diferentes tareas de manera que se puedan cumplir ciertas restricciones de tiempo en la ejecución del clasificador de movimientos.
- * Implementar un sistema de clasificación de movimientos en tiempo real en una tarjeta de desarrollo ESP32.

Materiales Necesarios.

HARDWARE:

- * 1 ESP32 DEVKIT V1. (Tarjeta o placa de programación)
- * 1 MPU6050. (Sensor acelerómetro).
- * 1 OLED 0.96 inch SSD1306 128x32 4 pin I2C (Display) OPCIONAL PARA ESTA ACTIVIDAD.
- * 1 juego cables (jumpers).
- * 1 Protoboard.

SOFTWARE:

- * Visual Studio Code con plugin PlatformIO.
- * Python.

Enlaces adicionales de consulta:

- * [Tutorial ESP32-MPU6050 \(1\).](#)
- * [Tutorial ESP32-MPU6050 \(2\).](#)
- * [Tutoriales FreeRTOS.](#)

Marco Teórico.

Función de Distribución Acumulada (CDF): La CDF de una variable aleatoria es una función que, para cualquier valor dado, proporciona la probabilidad de que la variable aleatoria tome un valor menor

o igual a ese valor. En términos más simples, describe la distribución acumulada de probabilidades a lo largo de todos los posibles valores de la variable.

La regresión logística y el modelo probit son técnicas de clasificación similares pero con elementos diferentes. La regresión logística utiliza la función logística (sigmoide) como su CDF para modelar la probabilidad de un evento binario (por ejemplo, éxito/fallo, 1/0). El modelo Probit utiliza la función de distribución acumulada normal (CDF normal) para modelar las probabilidades. La CDF normal es la función acumulativa de la distribución normal (o gaussiana). La elección entre ambos modelos puede depender de la naturaleza de los datos y de las suposiciones que se puedan hacer sobre la distribución de los errores. Ambos modelos son valiosos y tienen aplicaciones prácticas en diversas áreas.

K-Nearest Neighbors (K-NN) es un algoritmo de aprendizaje supervisado utilizado tanto para problemas de clasificación como de regresión. Es uno de los algoritmos más simples y fáciles de entender en el campo del aprendizaje automático. Es fácil de entender e implementar y no hace suposiciones explícitas sobre la distribución de los datos. Sin embargo puede ser computacionalmente costoso para grandes conjuntos de datos, ya que requiere calcular las distancias a todos los puntos de datos de entrenamiento. El desempeño puede ser afectado por la escala de las variables, por lo que es recomendable normalizar o estandarizar los datos, y la elección del valor adecuado de K puede no ser trivial y requiere validación.

Para un sistema embebido, la elección entre implementar K-Nearest Neighbors (K-NN) o Regresión Logística depende de varios factores, incluyendo los requisitos de memoria, tiempo de ejecución y complejidad de implementación. A continuación, se presentan algunas características de cada algoritmo:

Probit: El modelo probit es un modelo de regresión lineal que se utiliza para modelar la probabilidad de un evento binario. A diferencia de la regresión logística, el modelo probit asume que los errores siguen una distribución normal estándar. Esto puede ser útil si los errores no siguen una distribución logística. El modelo probit también es fácil de interpretar, ya que los coeficientes se pueden interpretar como el cambio en la probabilidad de éxito por cada cambio unitario en la variable independiente. Además, el modelo probit es robusto a valores atípicos y puede manejar datos no lineales.

K-Nearest Neighbors (K-NN): Este algoritmo tiene la ventaja de ser simple y fácil de implementar. No requiere un proceso de entrenamiento complejo y simplemente almacena los datos de entrenamiento. Sin embargo, K-NN puede ser lento para conjuntos de datos grandes, ya que implica calcular la distancia a cada punto de entrenamiento para cada predicción. Además, K-NN necesita almacenar todos los datos de entrenamiento en la memoria, lo que puede ser problemático en dispositivos embebidos con recursos limitados.

Regresión Logística: La regresión logística es un algoritmo de clasificación lineal que es eficiente en el uso de la memoria y rápido en el tiempo de ejecución. Una vez entrenado, el modelo de regresión logística se puede representar con un pequeño conjunto de coeficientes y un intercepto, lo cual requiere mucha menos memoria que almacenar todos los datos de entrenamiento. La clasificación es muy rápida, ya que solo requiere una multiplicación matricial y una operación sigmoide (o softmax para múltiples clases), en lugar de calcular distancias para cada punto de datos de entrenamiento. Sin embargo, el

entrenamiento de un modelo de regresión logística puede ser computacionalmente intensivo. Por lo tanto, la regresión logística es generalmente preferible para sistemas embebidos debido a su eficiencia en el uso de la memoria y rapidez en el tiempo de ejecución. Puedes entrenar el modelo en una plataforma más potente y luego implementar solo el modelo entrenado en el dispositivo embebido.

Metodología.

El desarrollo de esta actividad es individual por lo que cada estudiante debe tener todos los materiales necesarios para su desarrollo. Se puede probar el algoritmo de la Función Probit inicialmente en Python y luego implementarlo en la ESP32. Puede hacer uso de herramientas como copilot o chatGPT para la generación de código en Python. Se sugiere el uso de la librería de scikit-learn para la implementación de la Función Probit y la Regresión Logística en python, pero en su implementación en ESP32, recuerde que no puede hacer uso de dichas librerías.

Como punto de partida se propone usar el siguiente código que es un ejemplo que permite obtener datos del acelerómetro y giróscopo de la ESP32. Los datos son transmitidos por el puerto serial desde la ESP32 al computador.

Programación de la ESP32 con la MPU6050 conectada para generar datos del acelerómetro y giróscopo y enviarlos por el puerto serial:

- * Realice la conexión física entre la ESP32 y el sensor MPU6050. Recuerde que debe conectar las señales VCC (del MPU6050 al ESP32), GND (del MPU6050 al ESP32), SCL (del MPU6050 al GPIO22 del ESP32) y SDA (del MPU6050 al GPIO21 del ESP32)
- * Cree un nuevo proyecto y utilice este código en la ESP32. No olvide configurar la velocidad de transmisión y usar las librerías necesarias. Verifique que puede obtener los datos del acelerómetro y giróscopo en la consola serial de su computador.

```
1  /*#####
2  # Este código fue tomado y modificado de las librerías de Adafruit.
3  #####*/
4
5  /*#####
6  # LIBRARIES.
7  #####*/
8  // Basic demo for accelerometer readings from Adafruit MPU6050
9
10 #include <Adafruit_MPU6050.h>
11 #include <Adafruit_Sensor.h>
12 #include <Wire.h>
13
14 // Objeto para el sensor MPU6050
15 Adafruit_MPU6050 mpu;
16
17 void setup(void) {
18   // Inicialización comunicación serial a 115200 bps
19   Serial.begin(115200);
20
21   // Esperar hasta que la consola serial este abierta
22   while (!Serial) {
23     delay(10); // will pause Zero, Leonardo, etc until serial console opens
24   }
```

```

26 // Inicialicion sensor MPU6050
27 // Try to initialize!
28 if (!mpu.begin()) {
29     Serial.println("Failed to find MPU6050 chip");
30     while (1) {
31         delay(10);
32     }
33 }

35 // Configuracion rango acelerometro a +/-2g
36 mpu.setAccelerometerRange(MPU6050_RANGE_2_G);

38 // Configuracion rango giroscopo a +/-500 grados por segundo
39 mpu.setGyroRange(MPU6050_RANGE_500_DEG);

41 // Configuracion ancho de banda filtro a 5 Hz
42 mpu.setFilterBandwidth(MPU6050_BAND_5_HZ);
43 Serial.println("");

45 delay(100);
46 }

48 void loop() {

50     /* Get new sensor events with the readings */
51     sensors_event_t a, g, temp;
52     mpu.getEvent(&a, &g, &temp);

54     /* Print out the values in format "accel_x;accel_y;accel_z;gyro_x;gyro_y;gyro_z;temp */
55     //Serial.print("AccelX:");
56     Serial.print(a.acceleration.x);
57     Serial.print(";");
58     //Serial.print("AccelY:");
59     Serial.print(a.acceleration.y);
60     Serial.print(";");
61     //Serial.print("AccelZ:");
62     Serial.print(a.acceleration.z);
63     Serial.print(";");
64     //Serial.print("GyroX:");
65     Serial.print(g.gyro.x);
66     Serial.print(";");
67     //Serial.print("GyroY:");
68     Serial.print(g.gyro.y);
69     Serial.print(";");
70     //Serial.print("GyroZ:");
71     Serial.print(g.gyro.z);
72     Serial.print(";");
73     //Serial.print("Temp:");
74     Serial.print(temp.temperature);
75     Serial.println("");

77     // Esperar 10 milisegundos antes de la proxima lectura. Frecuencia de muestreo f = 1 / ts donde ts = x/1000;
78     delay(x);
79 }

```

Script: data_acquisition.py: A continuación se presenta este script de python que permite leer los datos del puerto serial y almacenarlos en un archivo de texto. Este script se puede ejecutar en una terminal de linux/windows o en un entorno de desarrollo como Visual Studio Code. Ejecute el script y verifique que puede guardar los datos del acelerómetro y giróscopo en un archivo de texto en su computador. Este script permite guardar secuencialmente los valores del sensor diferenciando 4 movimientos y una condición de no movimiento, para crear una pequeña base de datos etiquetada. No olvide cerrar la consola serial en Visual Studio Code antes de usar este script de Python, dado que el puerto serial estaría abierto comunicando datos, y no puede ser usado por dos aplicaciones al mismo tiempo.

```

1 #####
2 # # Este código fue generado con la asistencia de ChatGPT de OpenAI.
3 #####
4
5 #####
6 # LIBRARIES.
7 #####
8 import serial
9 import time
10 import pandas as pd
11 import numpy as np
12
13 class esp32Communication:
14     def __init__(self, port='/dev/ttyUSB0', baudrate=115200, timeout=0.1):
15         self.esp32 = serial.Serial(port=port, baudrate=baudrate, timeout=timeout)
16
17     def read_port(self):
18         packet = self.esp32.readline()
19         return packet.decode('utf-8').rstrip('\n')
20
21     def close(self):
22         self.esp32.close()
23
24 def file_treat(data_receive):
25     data = np.array(data_receive)
26
27     # Asegurar que el tamaño total sea un múltiplo de 7
28     remainder = len(data) % 7
29     if remainder != 0:
30         data = data[:-remainder] # Eliminar los elementos extra
31
32     data_div = np.char.split(data, sep=';')
33     data_div = np.concatenate(data_div)
34     data_div = data_div.reshape((-1, 7)).astype(float)
35     col_names = ['Accel x', 'Accel y', 'Accel z', 'w_x', 'w_y', 'w_z', 'Temp']
36     df = pd.DataFrame(data_div, columns=col_names)
37     return df
38
39 def capture_data(category, duration=15):
40     esp32_comm = esp32Communication()
41     try:
42         data_list = []
43         start_time = time.time()
44         print(f"Capturando datos para la categoría '{category}' durante {duration} segundos...")
45         while (time.time() - start_time) < duration:
46             value = esp32_comm.read_port()
47             if value:
48                 data_list.append(value)
49                 time.sleep(0.1) # Ajusta el tiempo de espera según sea necesario
50     except KeyboardInterrupt:
51         print("\nProgram interrupted by user. Exiting...")
52     finally:
53         esp32_comm.close()
54
55     df = file_treat(data_list)
56     df['Category'] = category
57     return df
58
59 if __name__ == "__main__":
60     print("Bienvenido a la captura de datos de tus movimientos")
61     data_frames = []
62
63     # Diccionario de entrada con la categoría y duración en segundos
64     input_dict = {
65         'Mov00': 10,
66         'Mov01': 10,
67         'Mov02': 10,
68         'Mov03': 10,
69         'Mov04': 10,
70     }
71
72     for category, duration in input_dict.items():

```

```

73     print(f"Preparandote para capturar el movimiento '{category}'")
74     input("Presiona Enter para comenzar...")
75     df = capture_data(category, duration)
76     data_frames.append(df)
77
78     if data_frames:
79         final_df = pd.concat(data_frames, ignore_index=True)
80         csv_filename = 'data.csv'
81         final_df.to_csv(csv_filename, index=False)
82         print(f'Datos guardados en {csv_filename}')
83     else:
84         print("No se capturaron datos.")
85

```

Trabajo a realizar.

Captura de datos:

- * Conecte la ESP32 con el sensor MPU6050 y la pantalla OLED. Cargue el programa explicado en la sección anterior y verifique que se obtienen los datos del acelerómetro y giróscopo en la consola serial de su computador.
- * Ejecute el código de la ESP32 y el script de Python `data_acquisition.py` en su computador para capturar datos del acelerómetro y giroscopio para diferentes movimientos. Este script permite guardar secuencialmente los valores del sensor diferenciando 4 movimientos y una condición de no movimiento, para crear una pequeña base de datos etiquetada. Realice una captura de datos de los movimientos los cuales puede ser: Mov00 (quieto), Mov01 (hacia arriba), Mov02 (hacia abajo), Mov03 (hacia la izquierda), Mov04 (hacia la derecha) o movimientos propuestos por usted pero manteniendo el Mov00 (quieto) propuesto. Este Mov00 (quieto) o reposo se sugiere por su facilidad de reconocimiento y porque permite organizar el protocolo para validar el sistema. Los datos quedarán guardados en un archivo csv con el nombre `data.csv`.

Clasificación de movimientos:

- * Implemente un clasificador utilizando la Función Probit para diferenciar los movimientos. Puede hacer pruebas inicialmente en python o cualquier lenguaje que domine. Utilice una Regresión Logística en caso de no tener buenos resultados solamente con la Función Probit.
- * Realice pruebas controladas con los archivos csv de datos y genere un nuevo archivo de texto con los resultados para cada secuencia de datos, la probabilidad calculada y el movimiento reconocido.

Implementación clasificador en la ESP32 en tiempo real:

- * Implemente el clasificador de movimientos en la ESP32 usando freeRTOS. Recuerde usar las librerías necesarias para la comunicación con el acelerómetro y giróscopo, la configuración de las tareas y la comunicación entre ellas. **El clasificador debe quedar implementado en la ESP32, para reconocer los movimientos propuestos en tiempo real.** El programa debe tener como mínimo

las tareas de: **Captura de Datos, Normalización/Clasificación y Visualización** (mínimo debe tener 3 tareas). Se debe tener como mínimo el uso de un semáforo y una cola para la comunicación entre tareas.

- Realice pruebas de funcionamiento del clasificador de movimientos en la ESP32. Genere un protocolo de pruebas para validar el funcionamiento de su clasificador de movimientos. Protocolo se refiere a pruebas controladas, donde el sistema sepa el movimiento que se está haciendo y el reconocedor pueda identificarlo.
- Genere un archivo raw con la información de los datos del acelerómetro y giróscopo de la ESP32, un identificador para cada movimiento ejecutado, uno para el movimiento reconocido y la función de probabilidad calculada para cada movimiento.

Entregables:

- * Se propone realizar el siguiente protocolo de pruebas: se debe generar una rutina controlada por tiempo, donde se realicen repeticiones de cada movimiento durante 15 segundos en el siguiente orden: Mov00 (quieto), Mov01, Mov00 (quieto), Mov02, Mov00 (quieto), Mov03, Mov00 (quieto), Mov04, Mov00 (quieto). Cada movimiento se debe hacer durante 15 segundos. Luego se debe generar una secuencia aleatoria de movimientos incluido Mov00 (quieto), por periodos de 5 segundos. Esto durante 30 segundos.
- * Al final de la rutina se debe generar un archivo de texto con los resultados obtenidos data.csv. El archivo raw debe tener los datos del acelerómetro y giróscopo; el movimiento ejecutado, el movimiento reconocido y la probabilidad calculada para cada movimiento.
- * Entregue los archivos main.cpp (archivo del ESP32) y data.csv con el formato de la tabla presentada a continuación.

Feature1	Feature2	Feature3	Feature4	Feature5	Feature6	Feature7	Clase Real	Clase Predecida	Prob Mov00	Prob Mov01	Prob Mov02	Prob Mov03	Prob Mov04
-2.3	8.78	6.08	-1.04	-0.73	0.06	22.57	Mov04	Mov04	0.000148	0.081453	0.18385	0.17336	0.561203
-3.8	8.55	3.1	0.36	0.01	-0.03	23.54	Mov01	Mov02	0.034273	0.786841	0.01718	0.095739	0.066020

donde:

Feature1 = Accel x (m/s²)

Feature2 = Accel y (m/s²)

Feature3 = Accel z (m/s²)

Feature4 = w x (rad/s)

Feature5 = w y (rad/s)

Feature6 = w z (rad/s)

Feature7 = Temp (oC)

Clase Real = identificador del movimiento ejecutado

Clase Predecida = identificador del movimiento reconocido

Prob Mov00 = Probabilidad de reconocimiento del movimiento Mov00

Prob Mov01 = Probabilidad de reconocimiento del movimiento Mov01

Prob Mov02 = Probabilidad de reconocimiento del movimiento Mov02

Prob Mov03 = Probabilidad de reconocimiento del movimiento Mov03

Prob Mov04 = Probabilidad de reconocimiento del movimiento Mov04

Es importante poder llegar a una solución que permita diferenciar los movimientos de manera clara y precisa para poder avanzar en la implementación de un clasificador de movimientos más complejo que se realizará más adelante. Todo el aprendizaje adquirido en esta actividad será fundamental para el desarrollo de las siguientes actividades.